1. Introduction

- Mitrayana is a web-based platform designed to provide assistance, care, and support services to senior citizens.

- As people grow older, they face challenges in health, daily tasks, social engagement, and technology usage.

- This portal acts as a **bridge** between senior citizens, volunteers, and administrators to ensure that seniors can live with dignity, independence, and comfort.

2. Abstract

- Senior citizens often struggle with daily activities due to limited family support and unreliable assistance from neighbors or unorganized volunteers. This creates challenges in ensuring timely, safe, and transparent care.

- The **Mitrayana project** provides a centralized platform that connects senior citizens, service providers, and administrators.

- Senior citizens can raise requests for services such as medical support, grocery delivery, companionship, and emergency help. Service providers respond efficiently, while administrators oversee the system for authenticity and smooth functioning.

- Mitrayana aims to create a **trusted digital ecosystem** where senior citizens can live with dignity, independence, and confidence, knowing reliable help is always available.

3. Objectives

- To provide **easy access to services** for elderly people such as medical help, home assistance, and companionship.
- To create a **volunteer network** where willing individuals can register and offer their services.
- To empower **admins** with tools to manage users, assign services, and monitor the overall system.
- To ensure **secure communication, payments, and transparency** between all stakeholders.
- To promote **social inclusion** and reduce loneliness among senior citizens.

4. Project Profile

4.1 Existing System

- **Elderly people** depend on **family, neighbors, or local contacts** for daily assistance (medical needs, groceries, companionship, emergencies).
- **Manual dependency** → Help arranged via phone calls or personal visits.
- **Unorganized volunteer efforts** → No unified platform, causing delays and lack of trust.
- **No proper monitoring** → Requests, payments, and service quality are not tracked.
- **Lack of transparency** → Seniors and families cannot verify the credibility of helpers.

4.2 Proposed System

The **Mitrayana system** is designed to provide a **centralized, user-friendly, and reliable platform** that connects senior citizens, service providers, and administrators.

Unlike the existing unorganized methods, the proposed system ensures transparency, timely assistance, and proper monitoring of all service requests

Features of the Proposed System

1. **Role-Based Modules**
   - **Admin Module:** Manages users, monitors requests, verifies service providers, and ensures system security.
   - **Senior Citizen Module:** Allows seniors to request services, track requests, and provide feedback.
   - **Service Provider Module:** Enables volunteers or service providers to view requests, accept tasks, and update progress.
2. **Service Request Management**
   - Senior citizens can request services such as medical aid, grocery delivery, companionship, and emergency support.
   - Service providers respond efficiently through the platform.
3. **Transparent Monitoring and Tracking**
   - Admin can track all requests and services to ensure accountability.
   - Seniors can view the status of their requests in real-time.

4. **Feedback**
    - Seniors can provide feedback for completed services.
    - Helps maintain service quality and reliability.
5. **Secure and Reliable Platform**
    - All data is securely stored and role access is controlled.
    - Ensures trustworthy interactions among seniors, providers, and administrators.

4.3 Tools & Technologies

- **Frontend:** HTML5, CSS3, Js
- **Backend:** .NET/ASP.NET CORE
- **Database:** MySQL

5. *Modules*

5.1 Admin

- **User Management** – Register, verify, and manage Seniors & Volunteers.
- **Request Monitoring** – Manages service requests in real-time.
- **Volunteer Assignment** – Assign tasks based on skills, availability, and location.
- **Payment Oversigh**t – Manages and monitors secure transactions.
- **Report & Analytics** – Generate activity reports for transparency.
- **System Control** – Maintain overall platform settings & security.

5.2 Senior Citizen

- **Easy Registration & Login** – Simple and secure access for seniors.
- **Profile Management** – Update personal details, needs, and preferences.
- **Service Request** – Raise requests for health, household support, or a warm companionship.

- **Real-time Tracking** – Monitor the status of requests easily.
- **Secure Payments** – Make online payments for paid services.
- **Feedback System** – Share ratings and reviews for services received.

5.3 Service Provider

- **Easy Registration & Login** – Secure onboarding for volunteers/service providers.
- **Profile & Skills Management** – Add/update skills, availability, and location.
- **Request Notifications** – Get instant alerts for assigned service requests.
- **Task Handling** – Accept, update, and complete assigned tasks.
- **Real-time Updates** – Share progress/status of ongoing requests.
- **Earnings & Payments** – Track payments for completed services.
- **Feedback & Ratings** – Receive reviews from senior citizens. Can give their service experience too.

6. Feasibility Study

Introduction

- The feasibility study evaluates whether the proposed system, *Mitrayana – A Help Portal for the Elderly*, can be effectively designed, developed, and implemented using the available resources. It assesses the project from multiple perspectives, including technical, operational, economic, and ethical feasibility.

- The purpose is to ensure that the system is practical, affordable, and beneficial for both developers and end-users — senior citizens, service providers, and administrators.

6.1 Technical Feasibility

- The Mitrayana system is technically feasible as it utilizes a proven and accessible technology stack. The frontend is developed using **HTML, CSS, and JavaScript**, ensuring compatibility with all modern browsers. The backend is implemented using **ASP.NET** connected to a **MySQL 8.x database** for reliable data storage and management.

- All tools used in the project — such as **Visual Studio**, **MySQL Workbench** — are open-source or available as free educational licenses. The system can operate efficiently on standard computer hardware with minimal configuration. Since the portal is web-based, it does not require any platform-specific setup, making the technical implementation both cost-effective and easily deployable.

6.2 Operational Feasibility

Mitrayana is designed with user convenience at its core. It addresses real challenges faced by senior citizens in managing daily tasks and seeking timely help. The system provides:

- Simple, accessible dashboards for seniors, service providers, and administrators.
- Role-based access ensuring that each user interacts only with relevant modules.
- A help request system that efficiently connects seniors with available service providers based on location and pincode.
- A feedback module enabling users to rate their experiences, ensuring accountability.
- The intuitive interface, minimal form inputs, and larger visual elements make the system especially suitable for elderly users.

Hence, the system is operationally feasible and can be adopted in real-world environments with minimal training.

6.3 Economic Feasibility (Cost-Benefit Analysis)

The project is economically viable. Development costs are minimal, as the system relies on open-source technologies and free software tools. The only potential expenses involve hosting and internet connectivity. The benefits, however, are significant:

- Streamlined coordination between seniors and service providers.
- Reduced administrative workload.

- Enhanced communication and timely support for senior citizens.

In conclusion, the benefits far outweigh the costs, making Mitrayana economically feasible.

6.4 Schedule Feasibility

- The project was developed within the academic timeline allotted for the semester.
- The implementation was divided into systematic phases — requirement analysis, system design, database design, coding, testing, and documentation.
- Task distribution among team members ensured steady progress and timely completion.
- The development was completed successfully within the proposed schedule, confirming schedule feasibility.

6.5 Legal and Ethical Feasibility

- Given that Mitrayana handles sensitive personal data such as contact numbers and limited medical information, ethical considerations were prioritized during design and implementation.
- The system ensures that user data remains private and secure. Only authorized roles have access to relevant information. Since payments are handled via Cash on Delivery, risks related to online transactions are eliminated.

- Mitrayana adheres to basic principles of digital ethics — confidentiality, accessibility, and responsible data handling — making it legally and ethically sound for deployment

Conclusion

After evaluating all aspects — technical, operational, economic, schedule, and ethical — the *Mitrayana* system is found to be fully feasible. It can be implemented and expanded further with minimal risk, offering a practical and socially beneficial solution for assisting senior citizens with their daily needs.

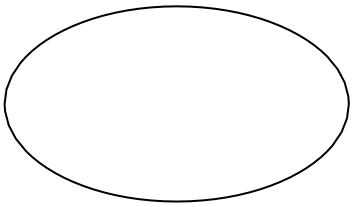
7. System Flow Diagram

Definition:

- A **System Flow Diagram (SFD)** is a graphical representation of the flow of data, processes, and decisions within a system.
- It shows how input is transformed into output through a sequence of processes, and how these processes are interconnected.
- SFDs help in understanding the logic, workflow, and structure of a system for analysis, design, or documentation purposes.

Shapes & Notations Explanation:

Terminal/Terminator:



Terminal/ Terminator

- Represents the start or end of the system/process.

  Example:

  Start: Senior submits help request
  End: Help request completed

Process/Activity:

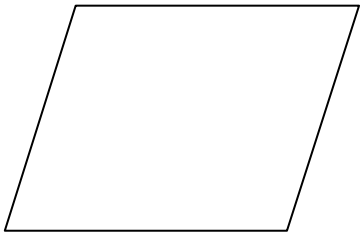
Process/Activity

- Represents a task, process, or operation performed.

Example:

Admin verifies volunteer details

Input /Output:



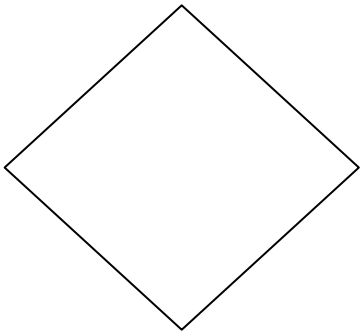
Input/ Output

- Represents data input into the system or output from the system.

Example:

Input: Senior fills request form
Output: Confirmation message

Decision:



Decision

- Represents a decision point that leads to different branches based on conditions.

  Example:

  Decision: Is volunteer available?
  Yes → Assign task
  No → Notify admin

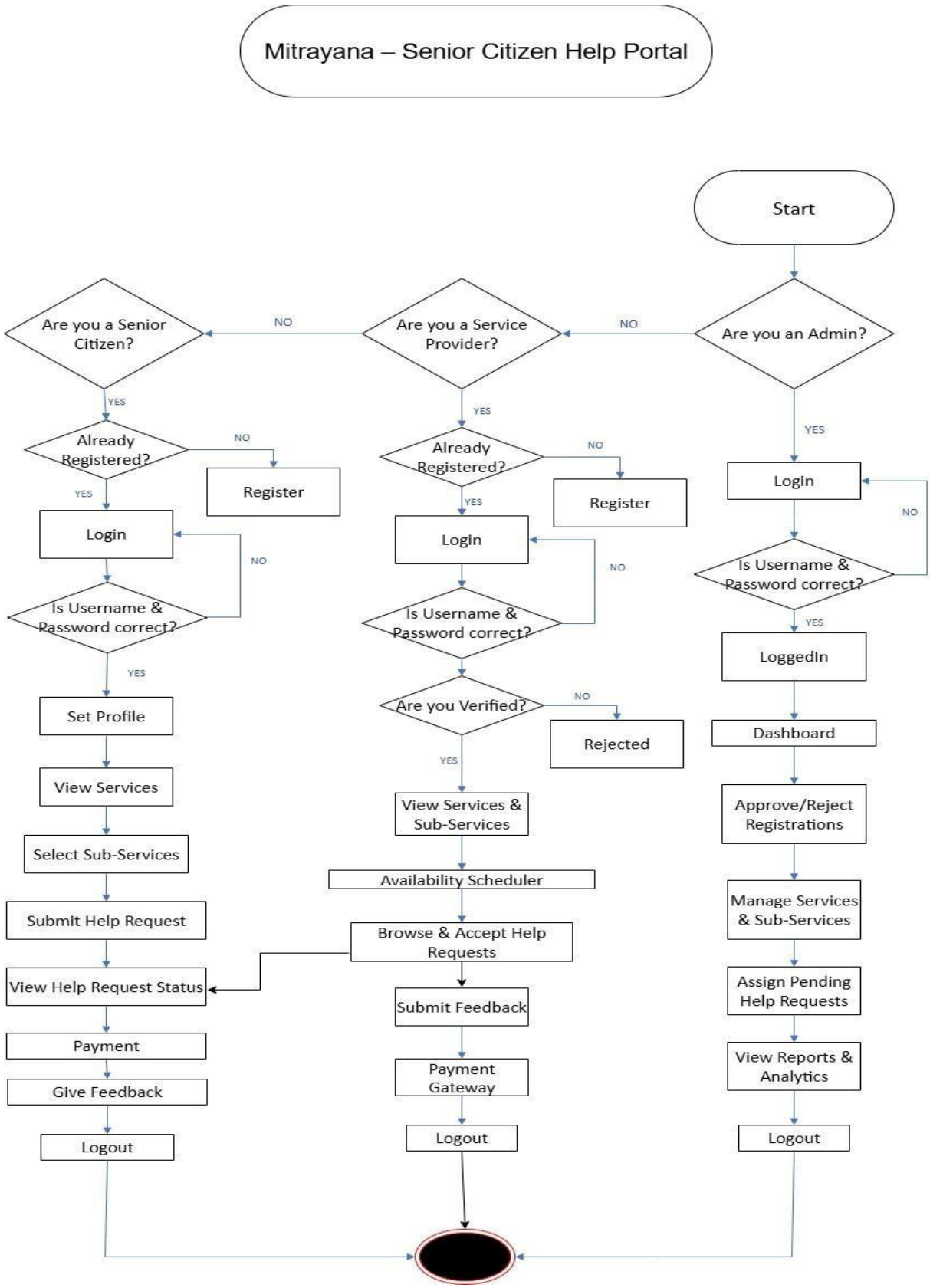
Flowline:

Flow Arrow

- Shows the direction of data or control flow between shapes.

  Example:

Connects senior request → admin verification → volunteer assignment


Mitrayana – Senior Citizen Help Portal
Start
Are you an Admin?
NO
YES
Login
Is Username & Password correct?
NO
YES
LoggedIn
Dashboard
Approve/Reject Registrations
Manage Services & Sub-Services
Assign Pending Help Requests
View Reports & Analytics
Logout
Are you a Service Provider?
NO
YES
Already Registered?
NO
Register
YES
Login
Is Username & Password correct?
NO
YES
Are you Verified?
NO
Rejected
YES
View Services & Sub-Services
Availability Scheduler
Browse & Accept Help Requests
Submit Feedback
Payment Gateway
Logout
Are you a Senior Citizen?
NO
YES
Already Registered?
NO
Register
YES
Login
Is Username & Password correct?
NO
YES
Set Profile
View Services
Select Sub-Services
Submit Help Request
View Help Request Status
Payment
Give Feedback
Logout

8. UML Diagram

8.1 Use Case Diagram

Definition:

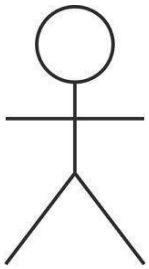
- A **Use Case Diagram** is a visual representation of how **actors** (users) interact with a system.

- It shows the **functionalities (use cases)** the system provides and the **roles** of different users.

- It helps in understanding **what the system does** (not how it works internally).

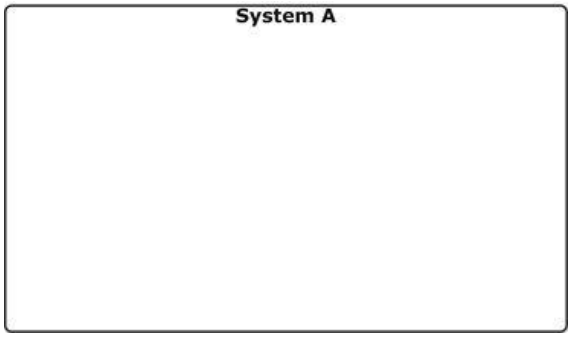
Shapes Explanation:

- **Actor (User/Role):**
  Drawn as a **stick figure** → represents a person or external system interacting with the application.
  (Example: Senior Citizen, Volunteer, Admin in Mitrayana).



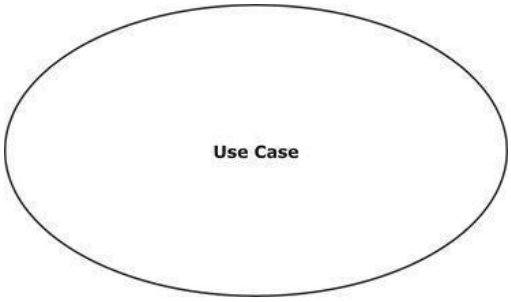
Actor

- **System Boundary:**
  Shown as a **rectangle box** → represents the whole system.
  (Example: Mitrayana Web Application).



System Boundary

- **Use Case (Function):**
  Drawn as an **oval shape** → represents a function/service provided by the system.
  (Example: Register, Request Help, Assist, Manage Users).

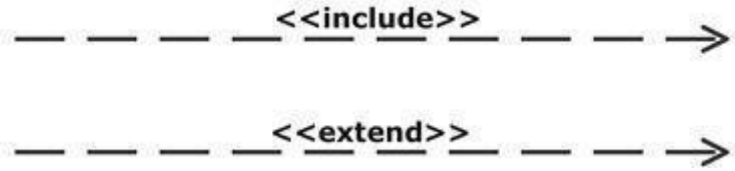


Function

- **Association Line:**
  A **straight line** connecting an actor to a use case → shows interaction between user and function.

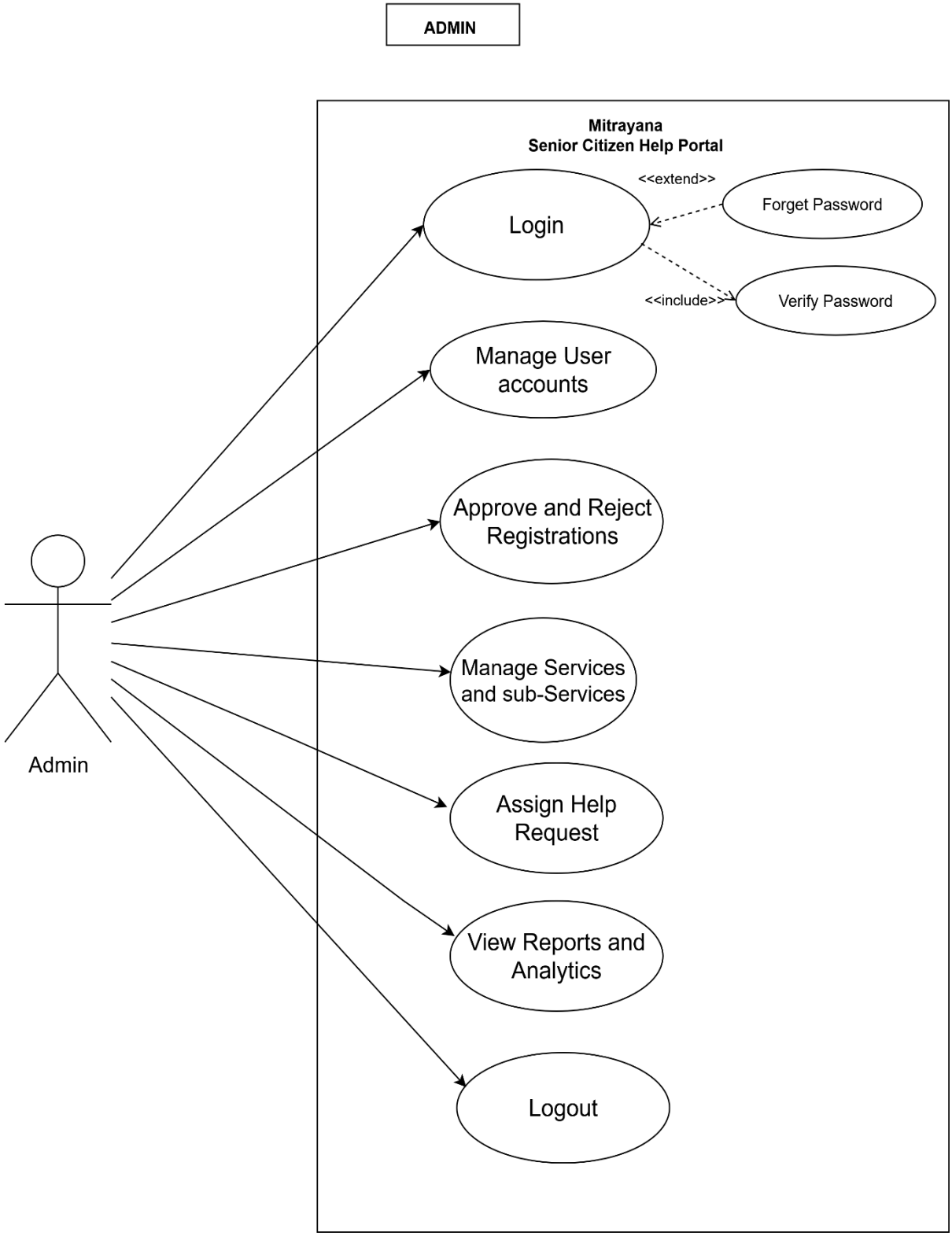
Association Line

- **Include / Extend (Optional):**
  - **<<include>>** → one use case always uses another.
  - **<<extend>>** → one use case sometimes adds extra behavior.

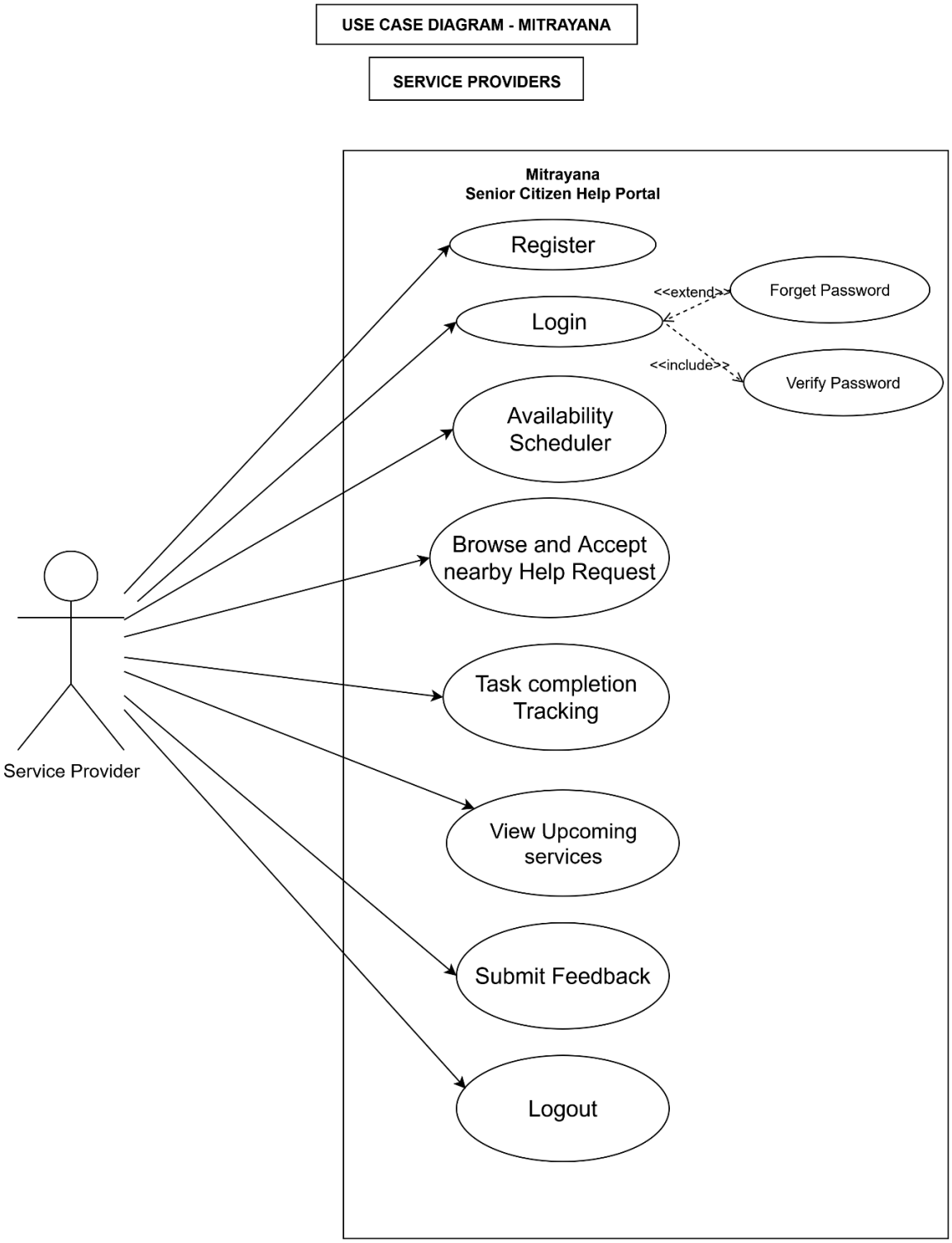


Include/Extend

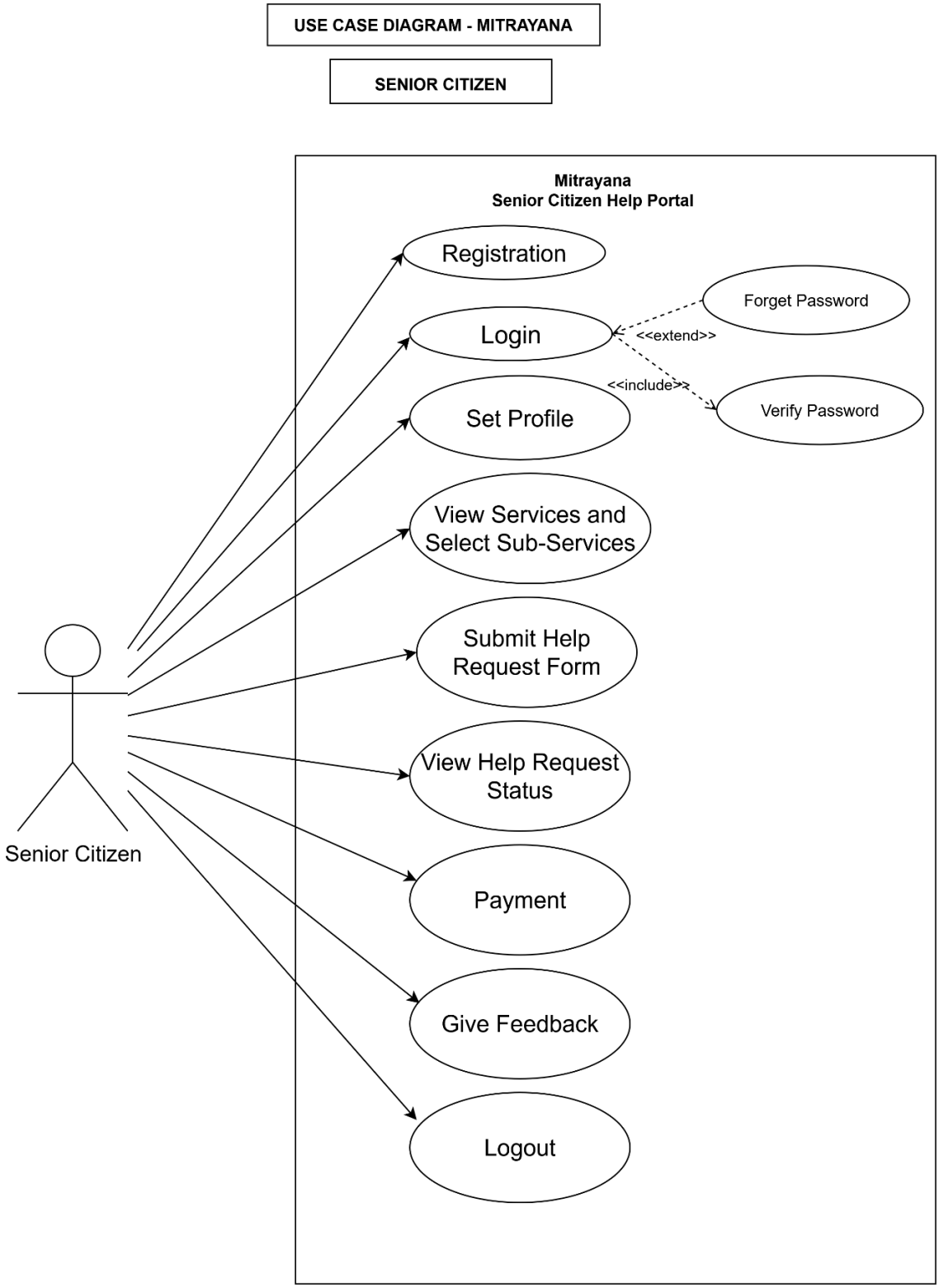
8.1.1 Admin Use Case Diagram



8.1.2 Service Provider Use Case Diagram



8.1.3 Senior Citizen Use Case Diagram



8.2 Sequence Diagram

Definition:

- A **sequence diagram** is a UML diagram that shows the interaction between objects or actors in a system in a time-ordered sequence.

- It is a UML diagram that illustrates how different objects communicate with each other to perform a specific function.

- They are mainly used to model use cases by representing the order of events between **actors and the system**.

Shapes & Notations Explanation:

1. **Actor**

- Represented as a stick figure.
- External entity that interacts with the system (e.g., **Senior Citizen, Volunteer, Admin).**

2. Object / Lifeline

- Drawn as a rectangle with the object name at the top.

- A vertical dashed line below the rectangle represents its lifeline (existence over time).

3. Message:

Relationship Lines in Class Diagram

Solid arrow → synchronous message (call, request).

Dashed arrow → return message (response).

Open arrow → asynchronous message (signal, event).

Represented by an arrow from one lifeline to another.

- Shows communication (method call, signal, or response).
- Can be synchronous (solid arrowhead) or asynchronous (open arrowhead).

4. Activation Bar:

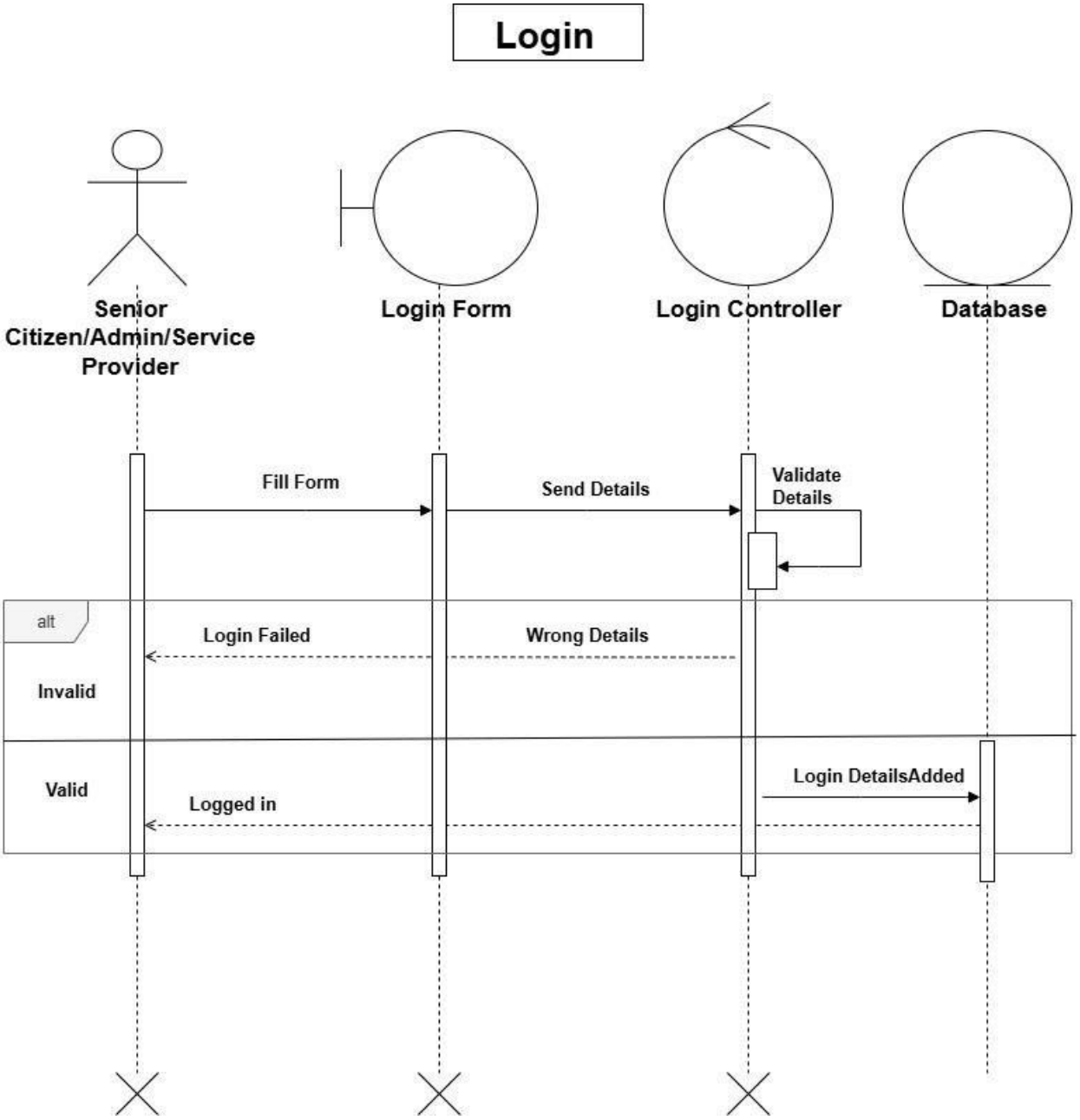
- A thin rectangle on a lifeline.
- Represents when an object is active (performing a task).

5. Destroy Symbol

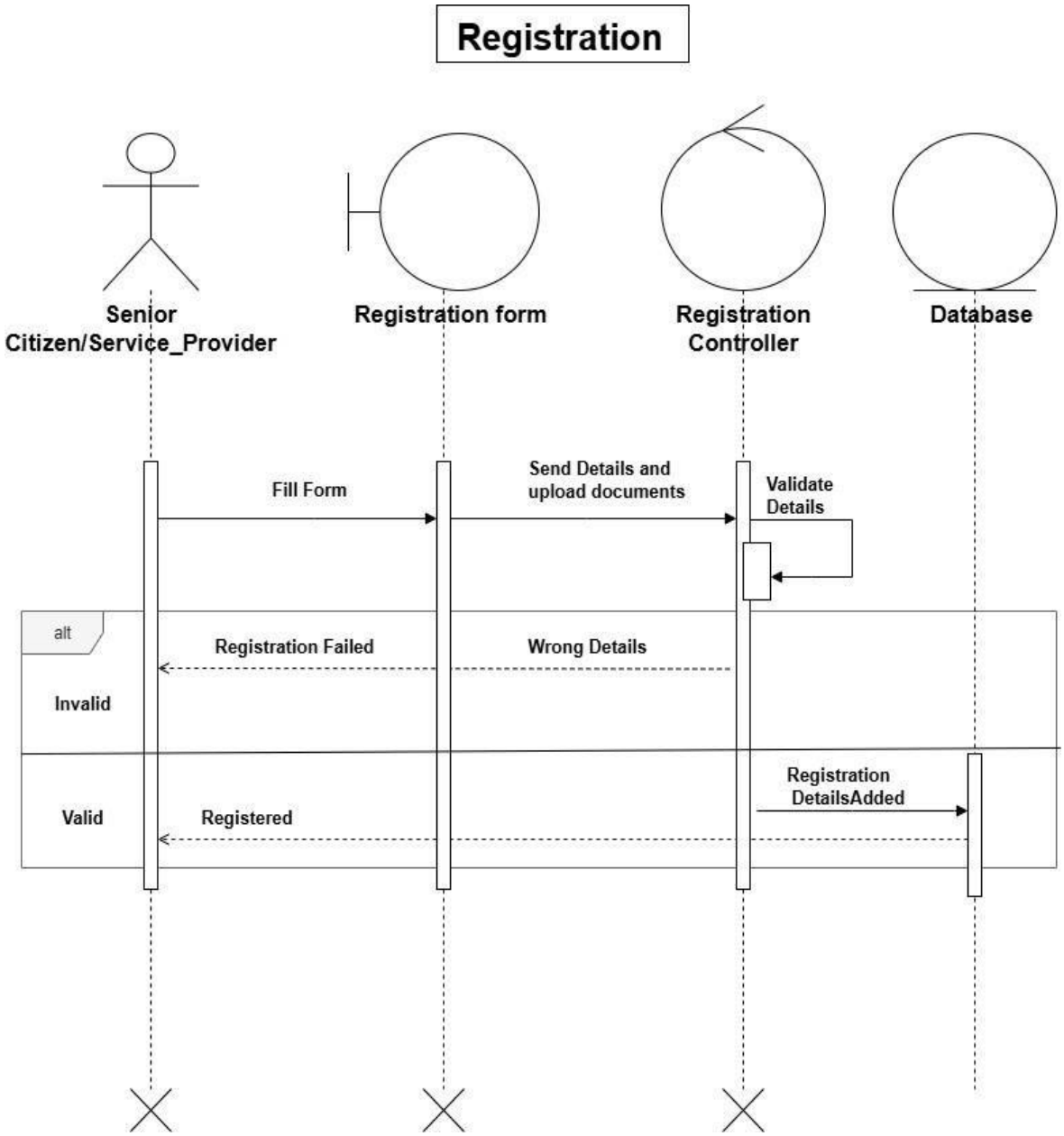
- A cross (X) at the end of a lifeline.
- Shows termination of an object in that scenario.

Sequence Diagram in Mitrayana:

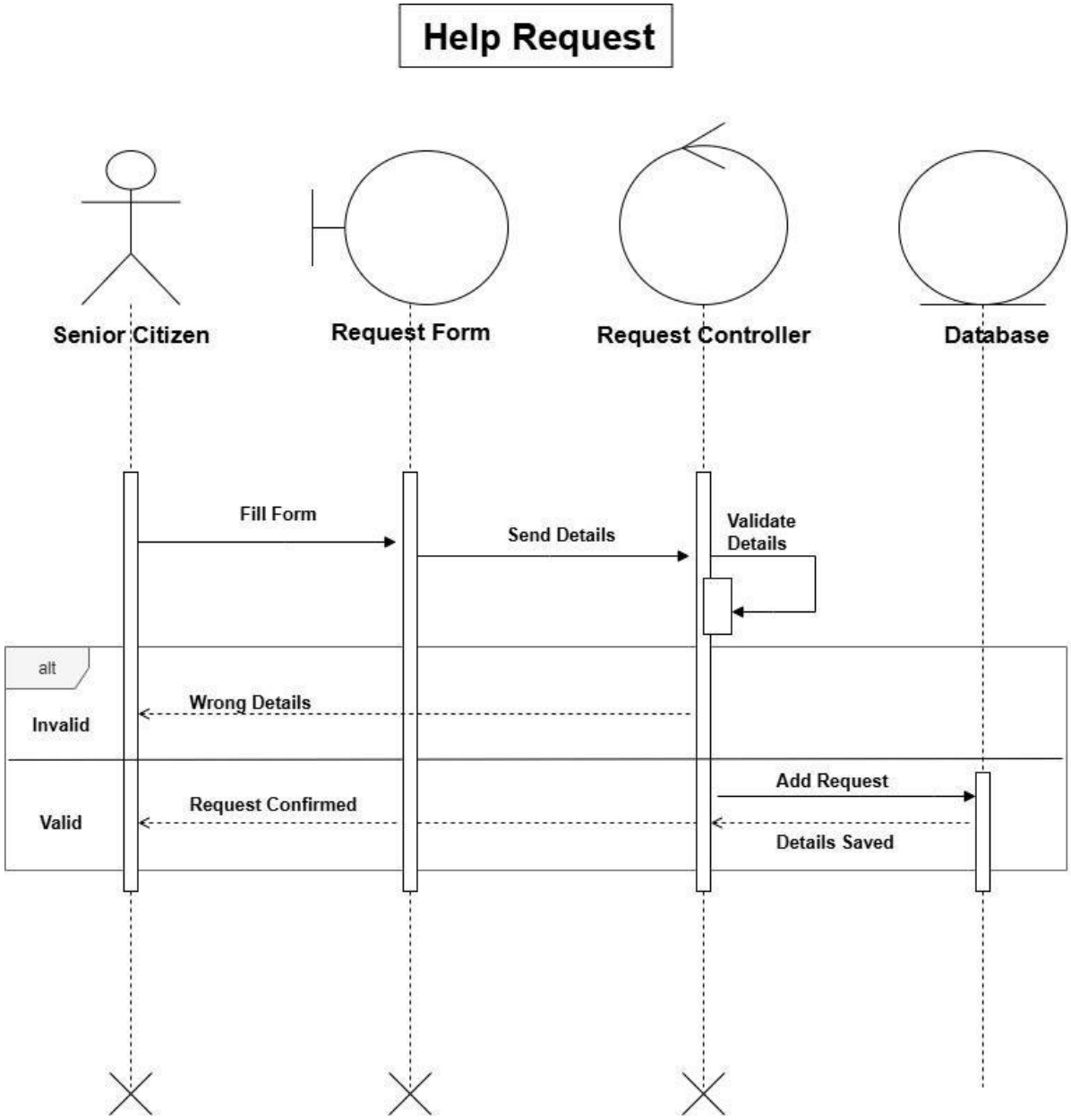
8.2.1 Login:



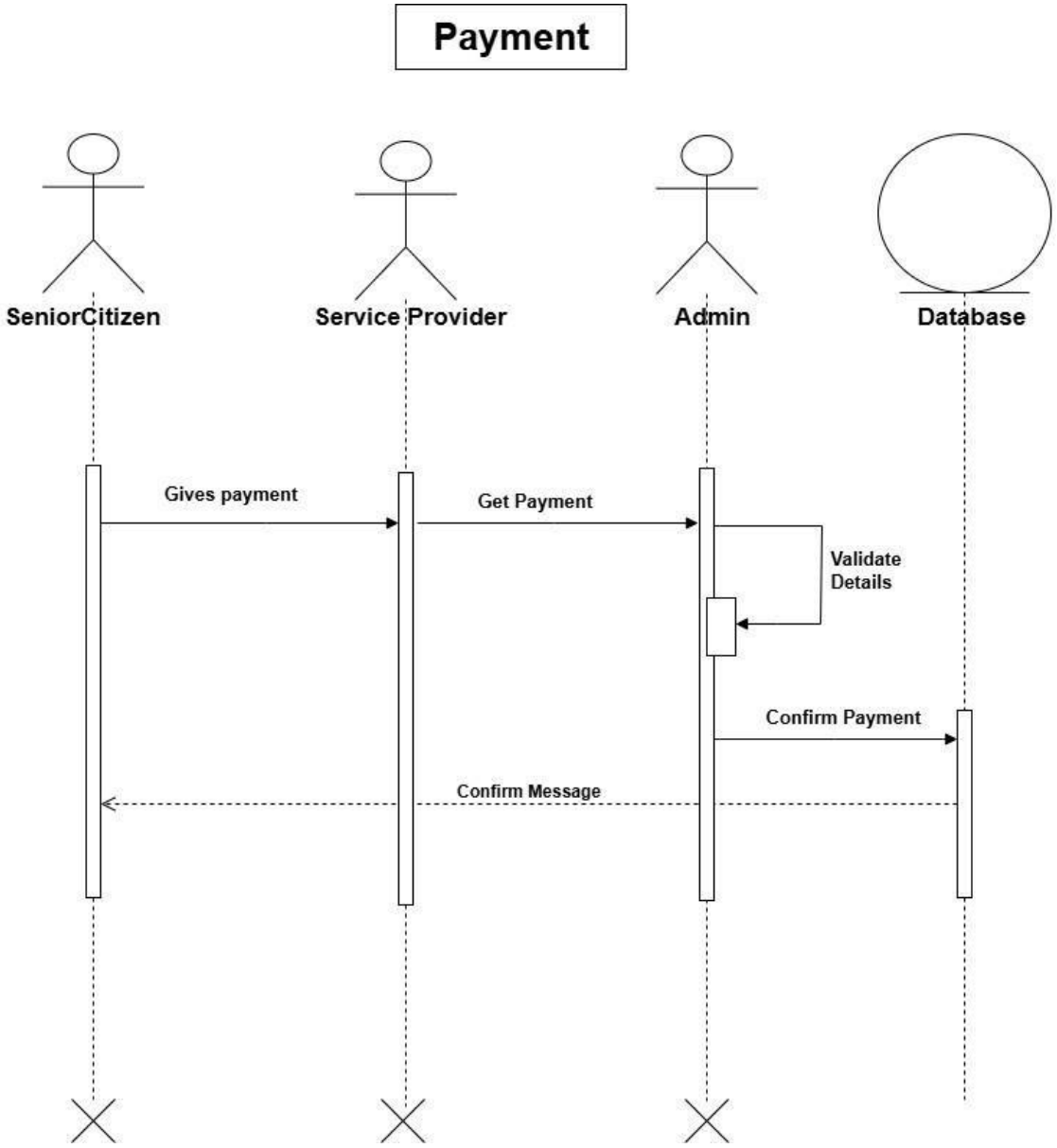
8.2.2. Registration



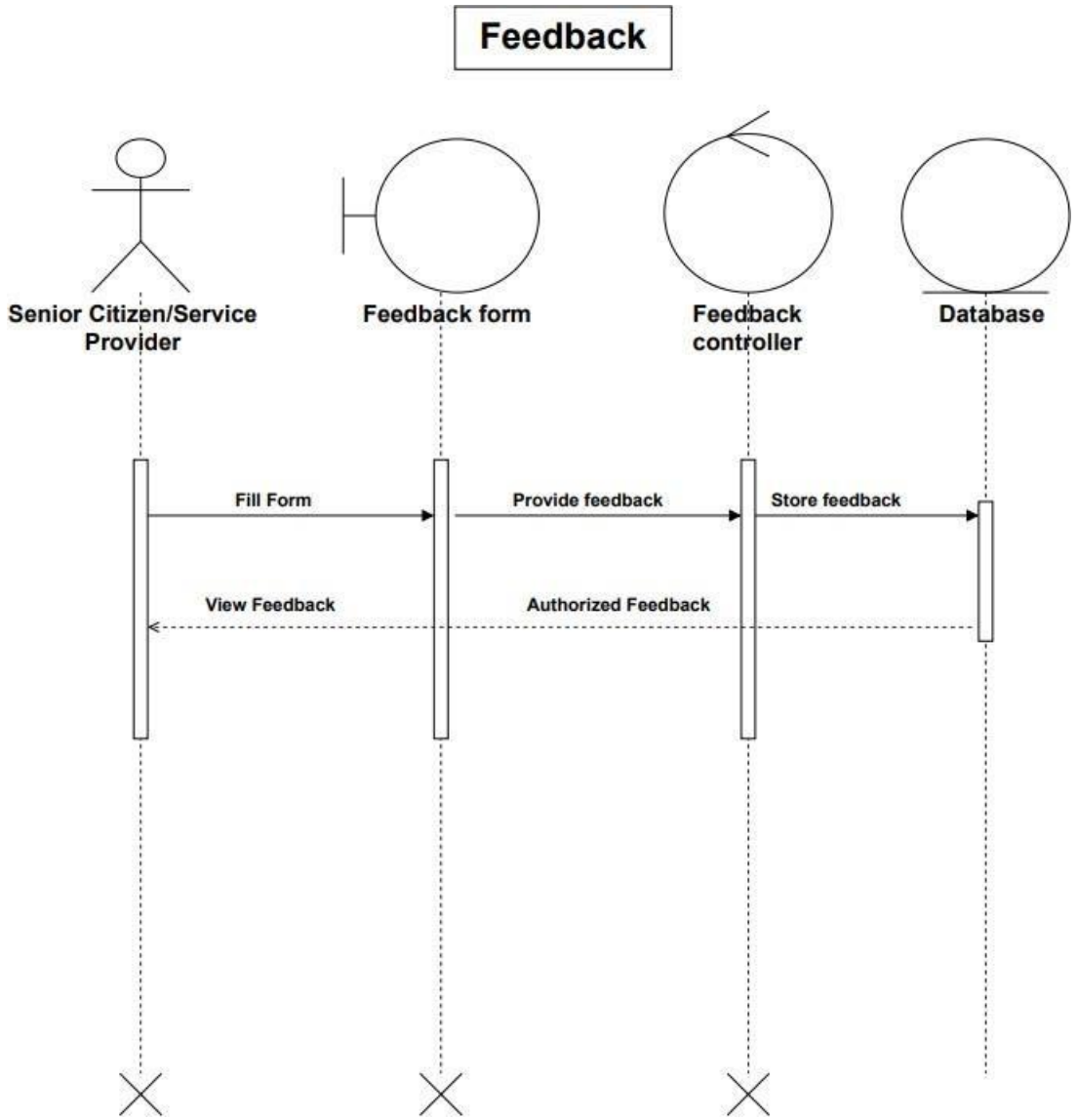
8.2.3 Help Request:



8.2.4 Payment


Payment
SeniorCitizen
Service Provider
Admin
Database
Gives payment
Get Payment
Validate Details
Confirm Payment
Confirm Message

8.2.5 Feedback



8.3 Activity Diagram

Definition:

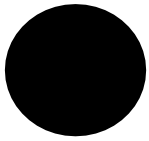
• An **Activity Diagram** is a UML diagram used to represent the **workflow of activities** in a system.

• It shows the **step-by-step execution flow** of processes, including **decisions, parallel tasks, and start/end points**.

• Useful for modeling both **business processes** and **system-level operations**.

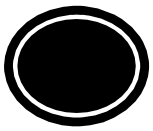
Shapes Explanation:

- **Initial Node:**
  Drawn as a filled black circle. → Indicates the **start of the workflow**.



Initial Node

- **Final Node:**
  Drawn as a black circle with a surrounding ring. → Indicates the **end of the workflow**.

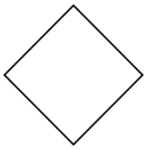


Final Node

- **Activity:**
  Shown as a **rounded rectangle** → Represents an action or task in the process.

Activity

- **Decision node:**
  Represented as a diamond shape. → Used for branching based on conditions

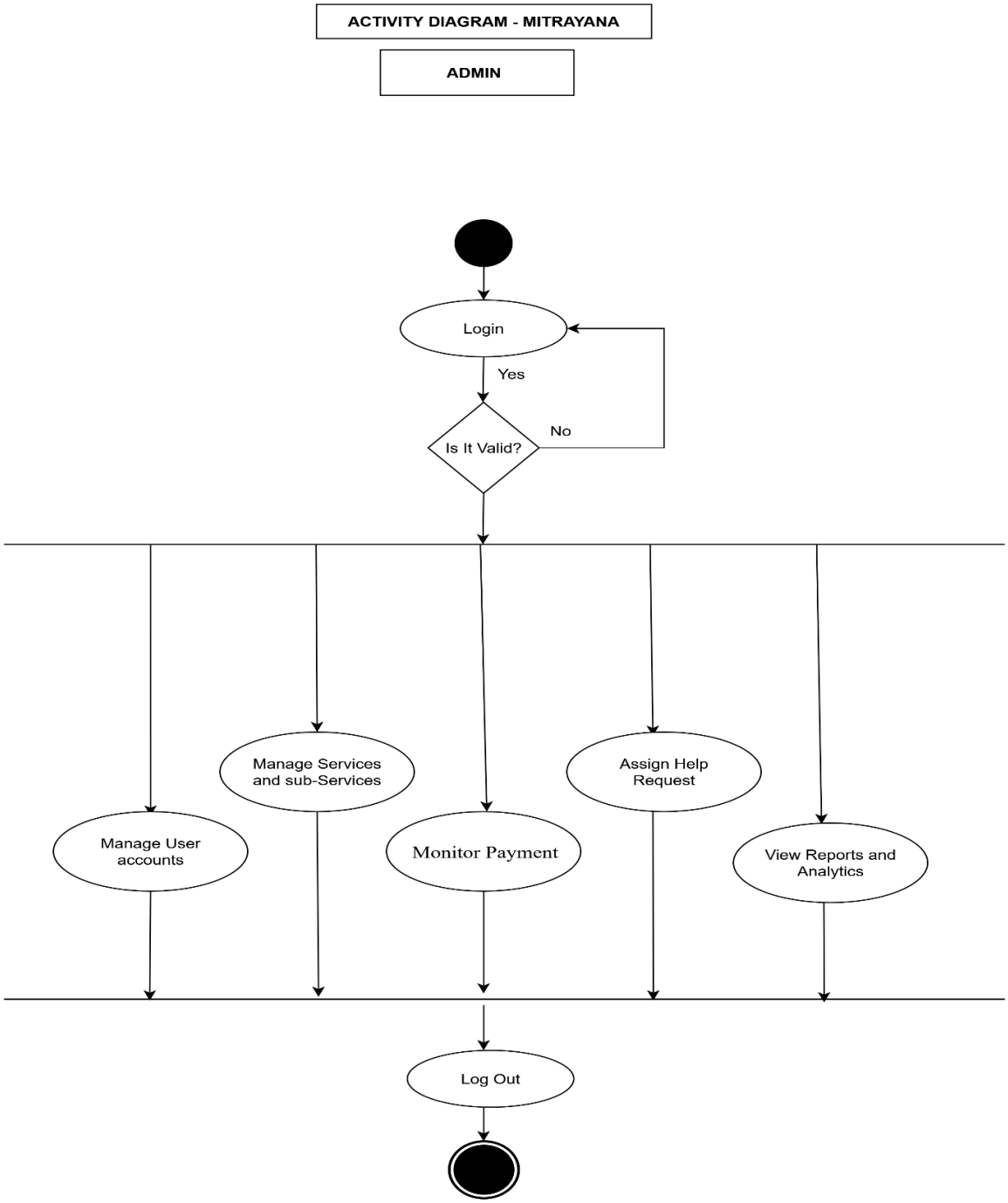


Decision Node

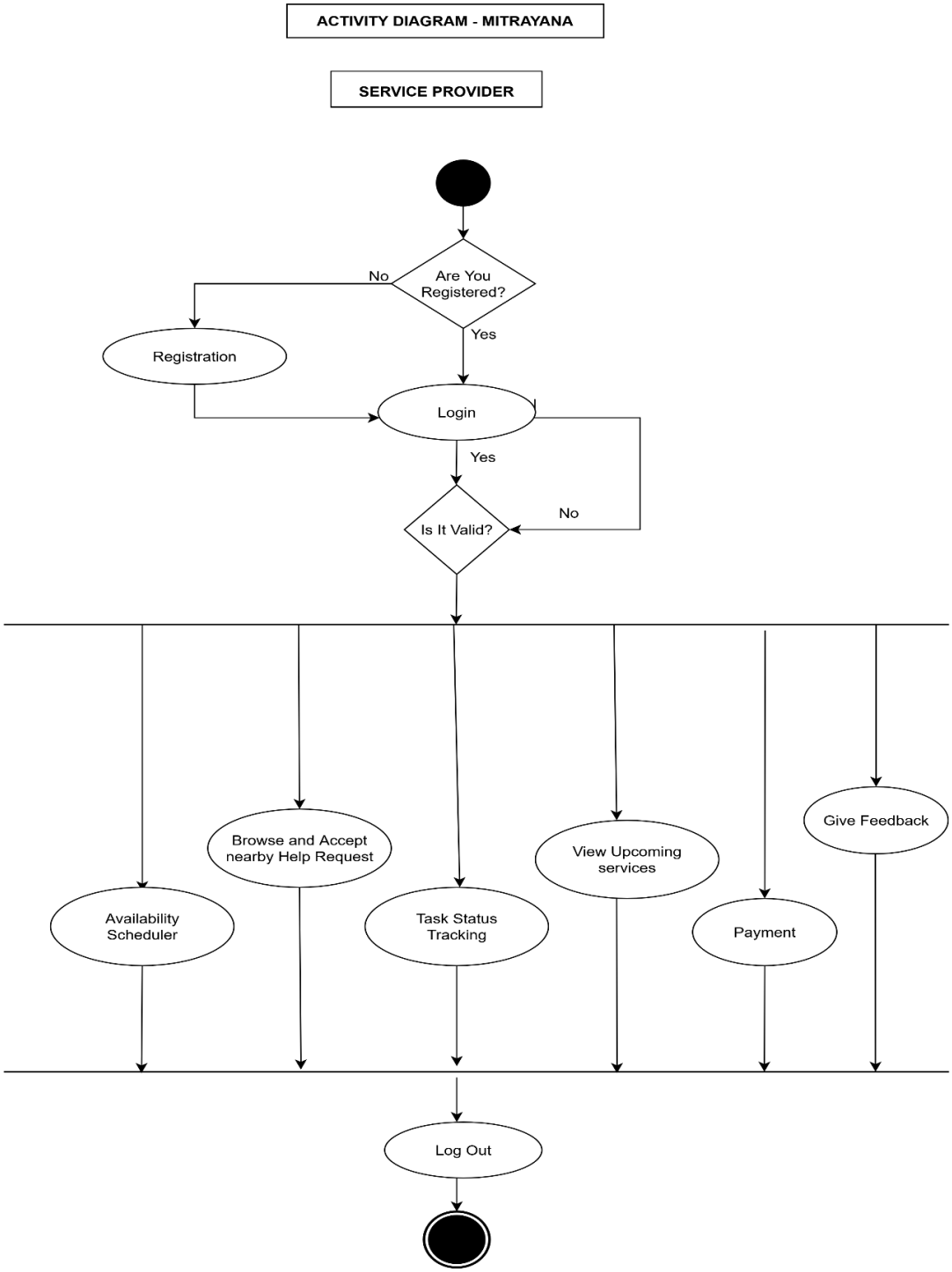
- **Control Flow / Arrow:**
  Arrows show the direction of execution from one activity to another.

Control Flow

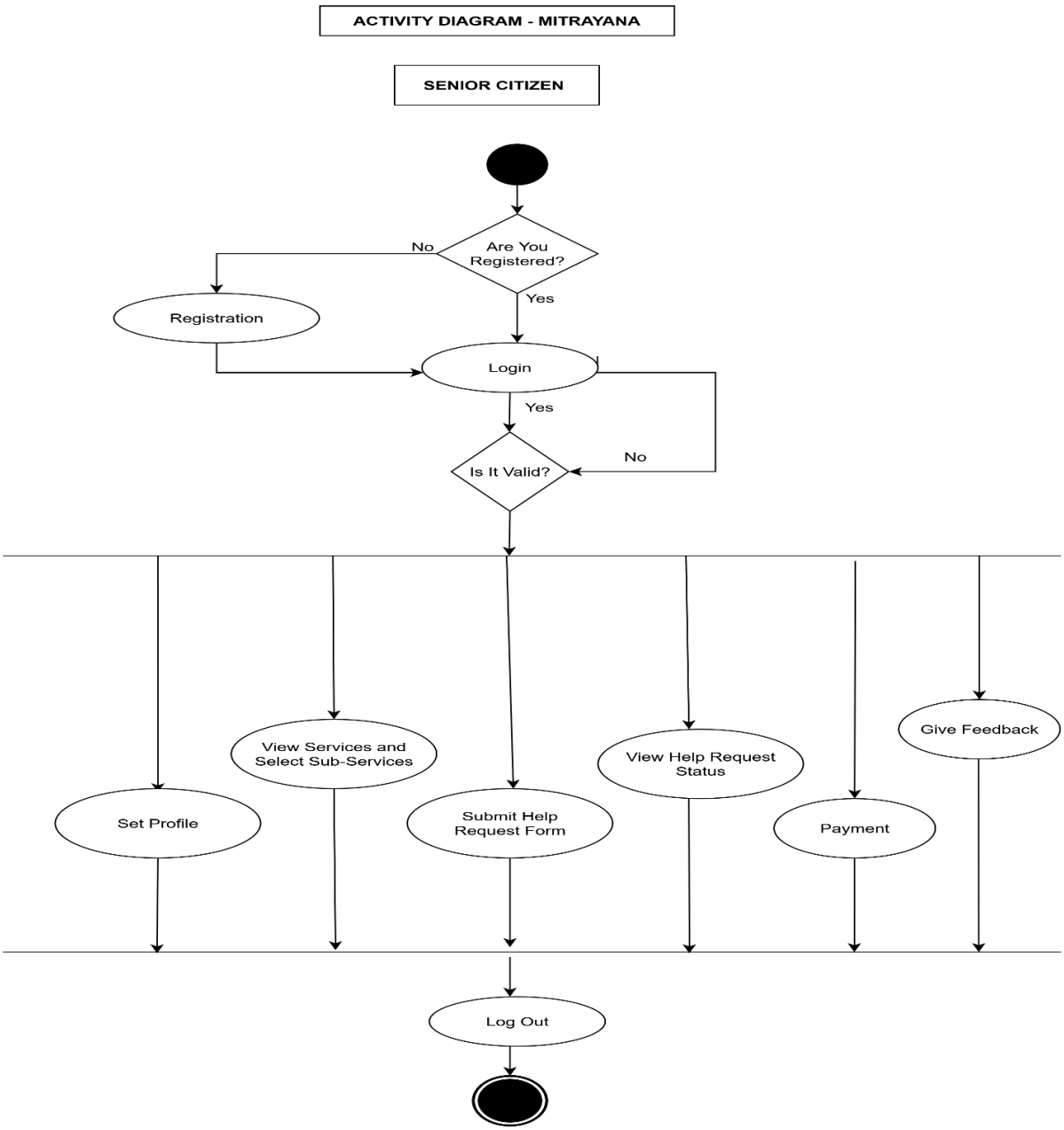
8.3.1Admin Activity Diagram



8.3.2 Service Provider Activity Diagram



8.3.3 Senior Citizen Activity Diagram



8.4Class Diagram

Definition:

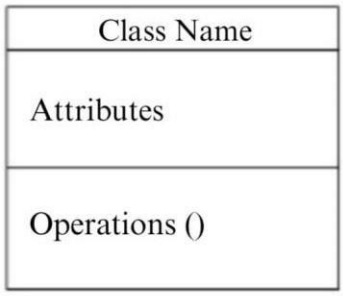
- A **Class Diagram** is a type of UML diagram that shows the **static structure of a system**.
- It represents **classes, their attributes, methods, and relationships**.
- It helps in understanding the **blueprint of the system** before coding.

Shapes & Notations Explanation:

Class (Rectangle Box):

- Drawn as a **3-part rectangle** →
  - **Top section:** Class name (e.g., `User`, `Senior Citizen`).
  - **Middle section:** Attributes (data members).
  - **Bottom section:** Methods (functions/operations).

- **Attributes:**
  - Shown with **data type** → (e.g., `Name: string`, `Age: int`).
  - Represent the **properties of the class**.
- **Methods (Functions):**
  - Shown with **()** → (*e.g.*, *`Register()`*, *`Login()`*).
  - Represent the **behaviors of the class**.

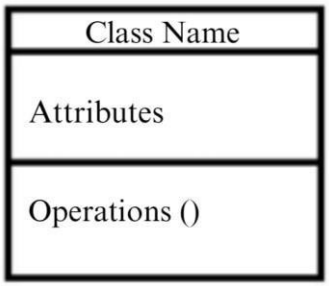


Active Class

An **Active Class** is a class whose objects have their **own thread of control** (they execute independently).

- Drawn as a **rectangle with thick borders**.
- **Mitrayana Example:**

  Help Request can be considered an Active Class because it **manages its own lifecycle** (Pending → Assigned → Completed) and continuously interacts with Senior Citizens and Service Providers.



Visibility in Class Diagram

Visibility specifies the **access level** of attributes and methods.

- **+ Public:** Accessible from anywhere.

  (e.g., `+ Register()`)

- **- Private:** Accessible only within the same class.

  (*e.g.*, `- Password:string`)

- **# Protected:** Accessible within the same class and its subclasses.

Multiplicity (Relations Between Objects)

- **1..1 (One-to-One):** One object of Class A is associated with exactly one object of Class B.
  - *Example (Mitrayana):* Each `Payment` is linked to **one Help Request only**.
- *1.. (One-to-Many):** One object of Class A is associated with many objects of Class B.
  - *Example (Mitrayana):* One `Senior Citizen` can create **many Help Requests**.

Relationship Lines in Class Diagram

1. **Association**
   - **Line:** Solid line with arrow
   - **Meaning:** Basic link between two classes
   - **Mitrayana Example:** `Senior Citizen` —▶ `Help Request`

Association

2. **Inheritance (Generalization)**
   - **Line:** Solid line with hollow triangle
   - **Meaning:** "Is-a" relationship (child inherits from parent)

- **Meaning:** A class implements an interface
- **Mitrayana Example:** `Payment` could implement a `PaymentInterface`

Inheritance

3. **Dependency**
   - **Line:** Dashed line with arrow
   - **Meaning:** Temporary usage in a method
   - **Mitrayana Example:** `Admin` depends on `Service` to update/remove services

Dependency

4. **Aggregation**
   - **Line:** Solid line with hollow diamond
   - **Meaning:** "Has-a" relationship, objects are independent
   - **Mitrayana Example:** `Feedback` ◇— `User` (Feedback refers to Senior Citizen and Volunteer, but Users exist independently)

Aggregation

5. **Composition**
   - **Line:** Solid line with filled diamond
   - **Meaning:** Strong "Has-a" relationship, ownership

- **Mitrayana Example:** `Help Request` ◆— `Request Details` (If Help Request is deleted, its details are also deleted)
- **Mitrayana Example:** `Senior Citizen`, `Service Provider`, `Admin` inherit from `User`

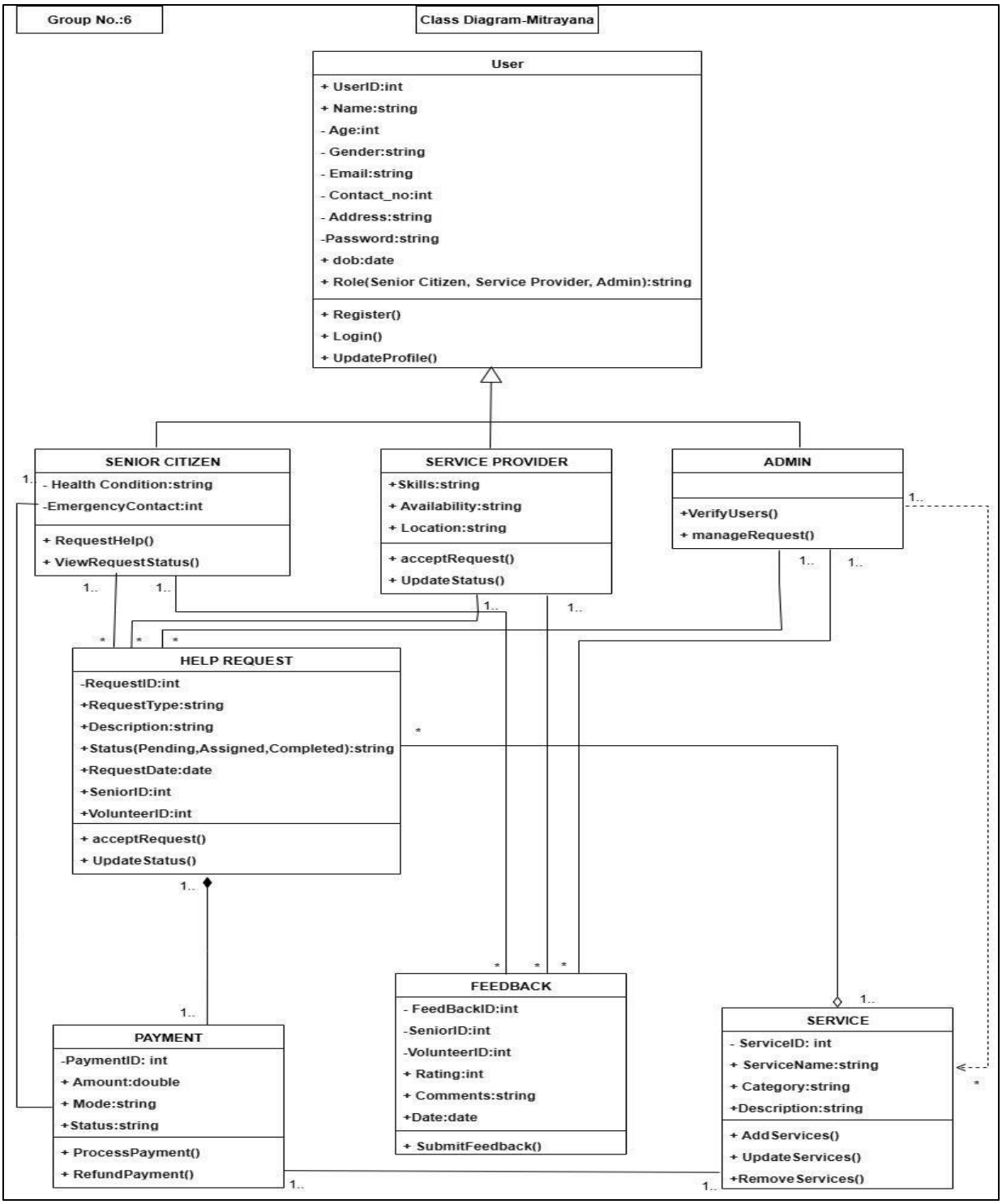
Composition

6. **Realization / Implementation**

- **Line:** Dashed line with hollow triangle
- **Meaning:** A class implements an interface
- **Mitrayana Example:** `Payment` could implement a `PaymentInterface`

Realization/Implementation

Class Diagram



8.5Object Diagram

Definition:

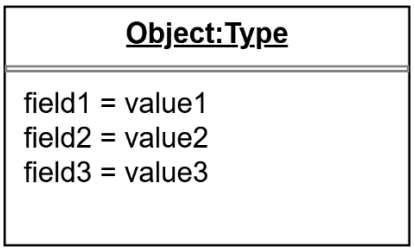
- An **Object Diagram** is a UML diagram that shows a **snapshot of objects and their relationships** at a particular point in time.
- It is similar to a **Class Diagram**, but instead of classes, it uses **instances (objects)** with actual data values.
- Helps visualize how system entities interact in a specific scenario.

Shapes & Notations Explanation:

Object:
Drawn as a **rectangle** with the object name underlined.

Format: ***objectName : ClassName***

(*Example: S1: SeniorCitizen)*.



Object

- **Links (Association among objects):**
  Shows an instance-level relationship → Represented by a **solid line** connecting two objects.

Link

- **Attributes and Values:**
  Inside the object box, attributes of the object are listed along with their specific values.

Format: objectName : ClassName

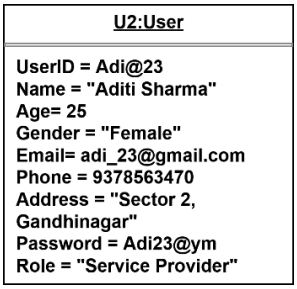
---

attribute1 = value1

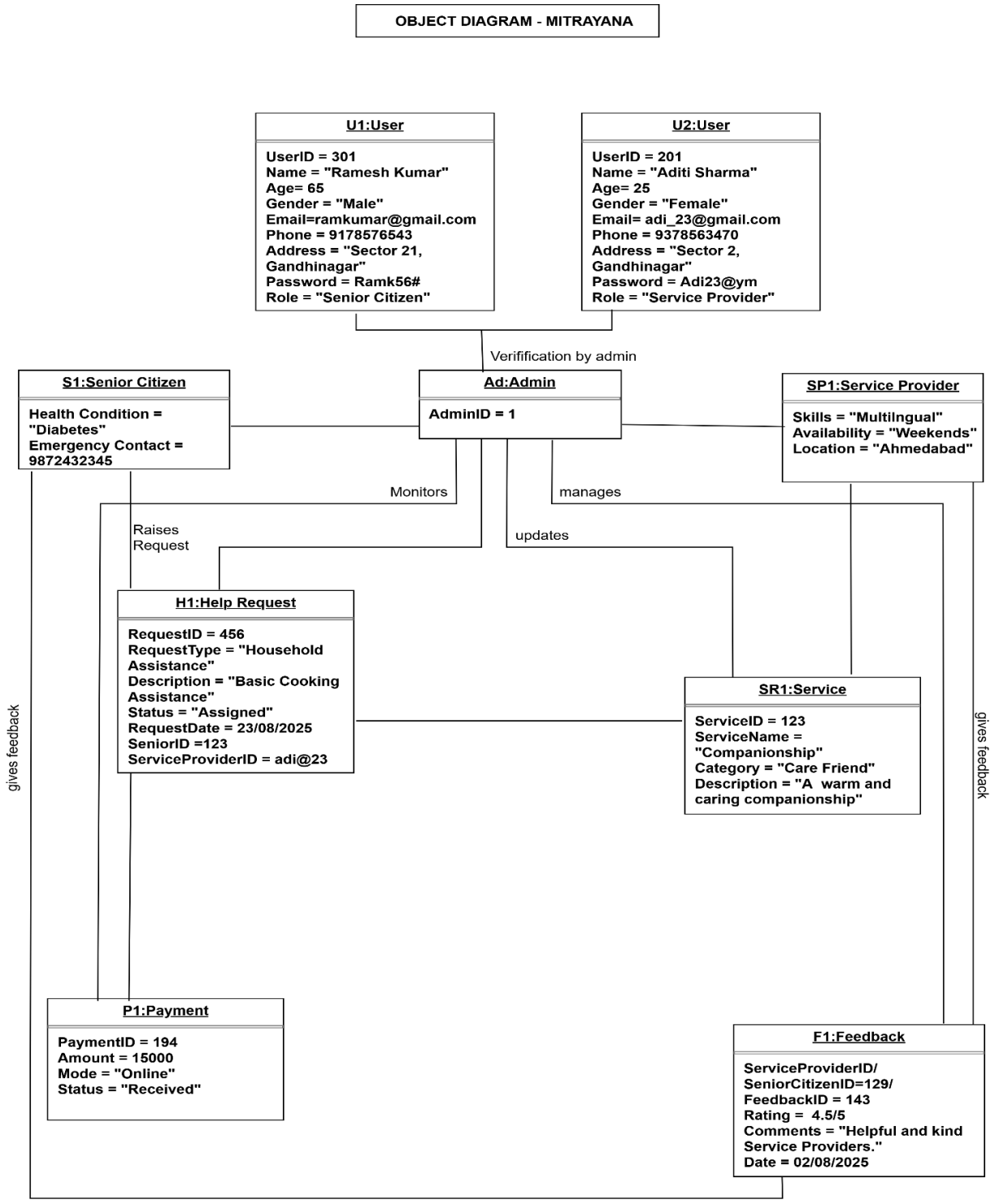
attribute2 = value2

attribute3 = value3

Example: User2 in Mitrayana web Application



Object Diagram



8.6 State Chart Diagram

Definition:

• A **State Chart Diagram** (or State Machine Diagram) shows the **lifecycle of an object** in the system.

• It explains how the object moves from one state to another in response to **events** or **conditions**.

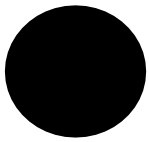
• It helps in understanding the **dynamic behavior** of the system (how it changes over time).

Shapes Explanation:

- **State:**
  Drawn as a **rounded rectangle** → represents a condition or situation during the life of an object.
  (Example: Registered, User management, Service provider's assignment in Mitrayana).

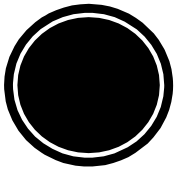
State

- **Initial State:**
  Drawn as a filled black circle. → represents the starting point of the diagram.



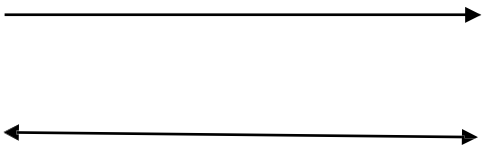
Initial State

- **Final State:**
  Drawn as a black circle with a surrounding ring. → represents the end of the lifecycle.



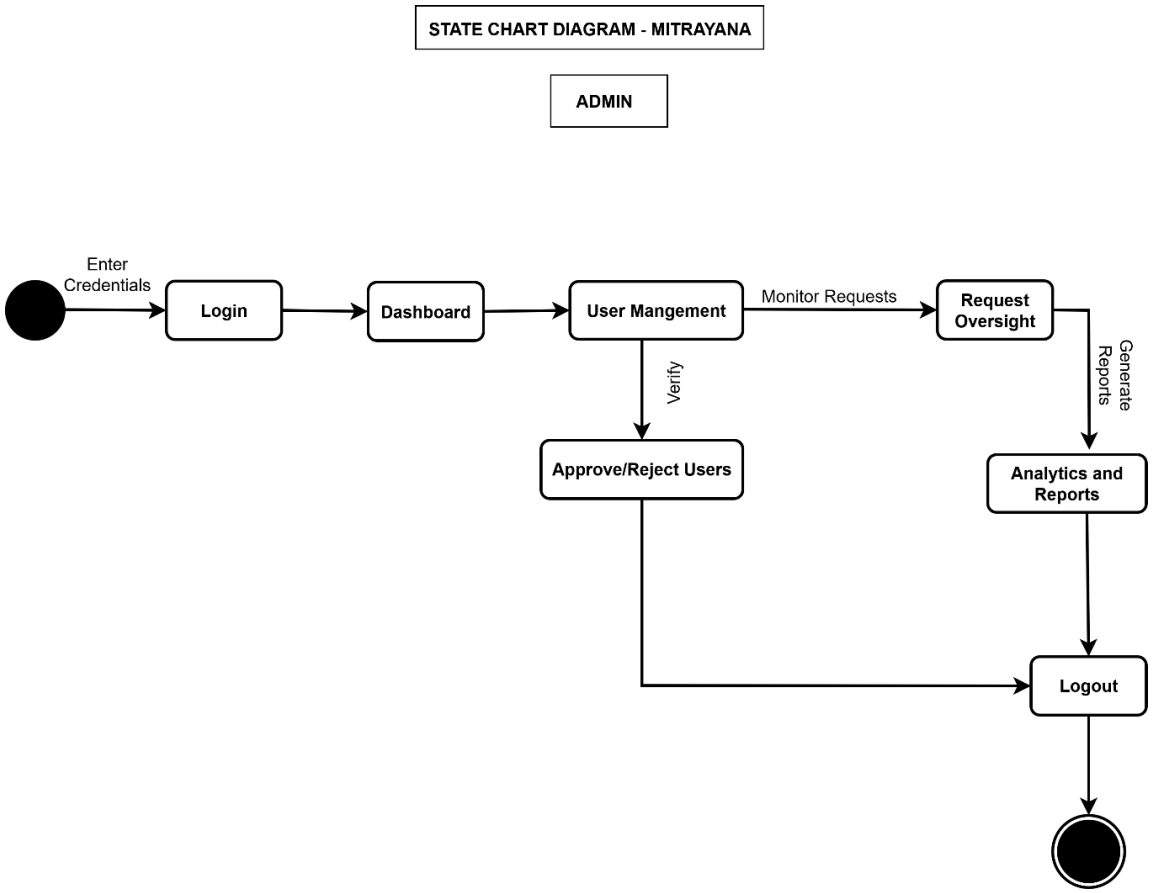
Final State

- **Transition:**
  Shown as an **arrow** between states→ Labeled with the event/action that causes the change

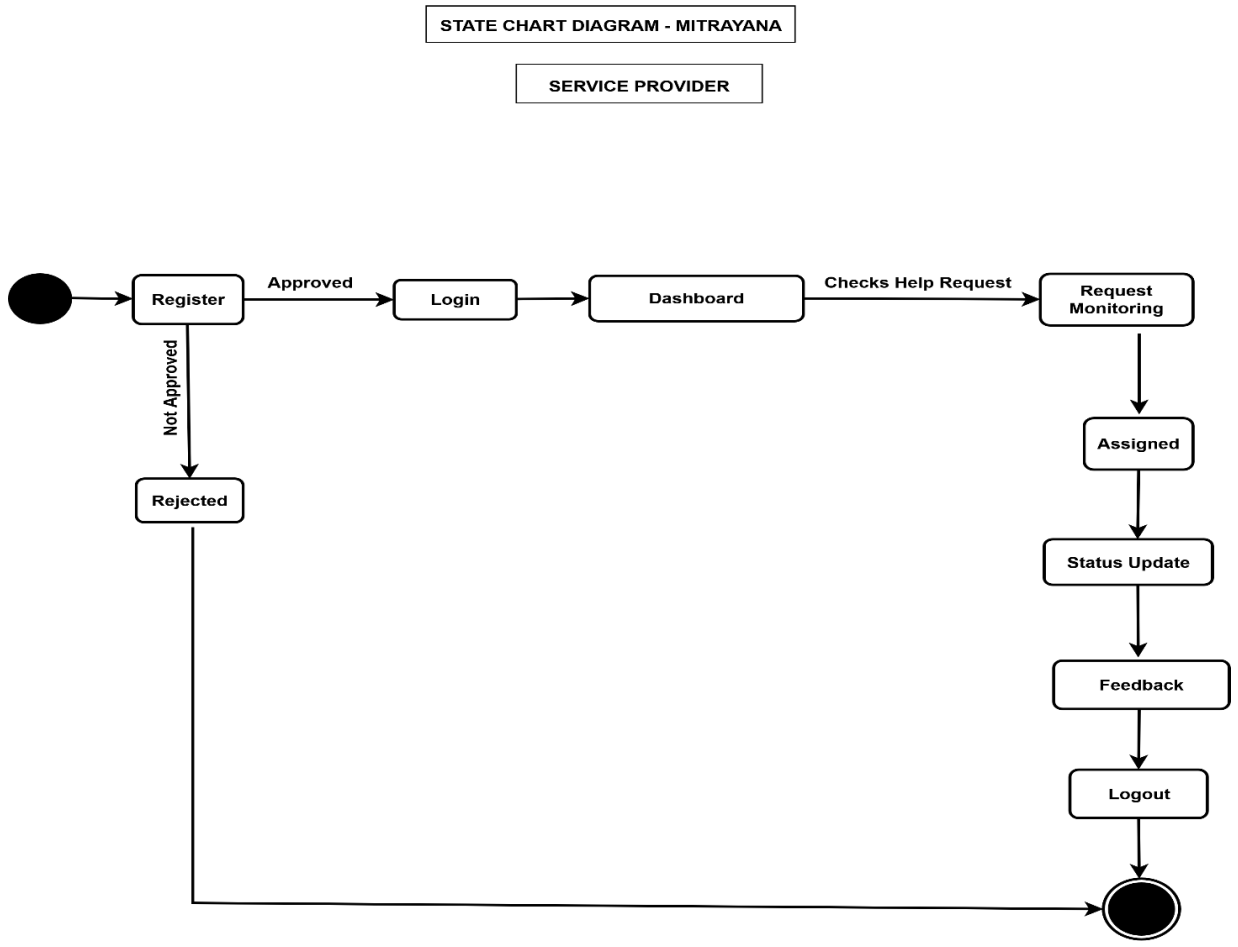


Transition

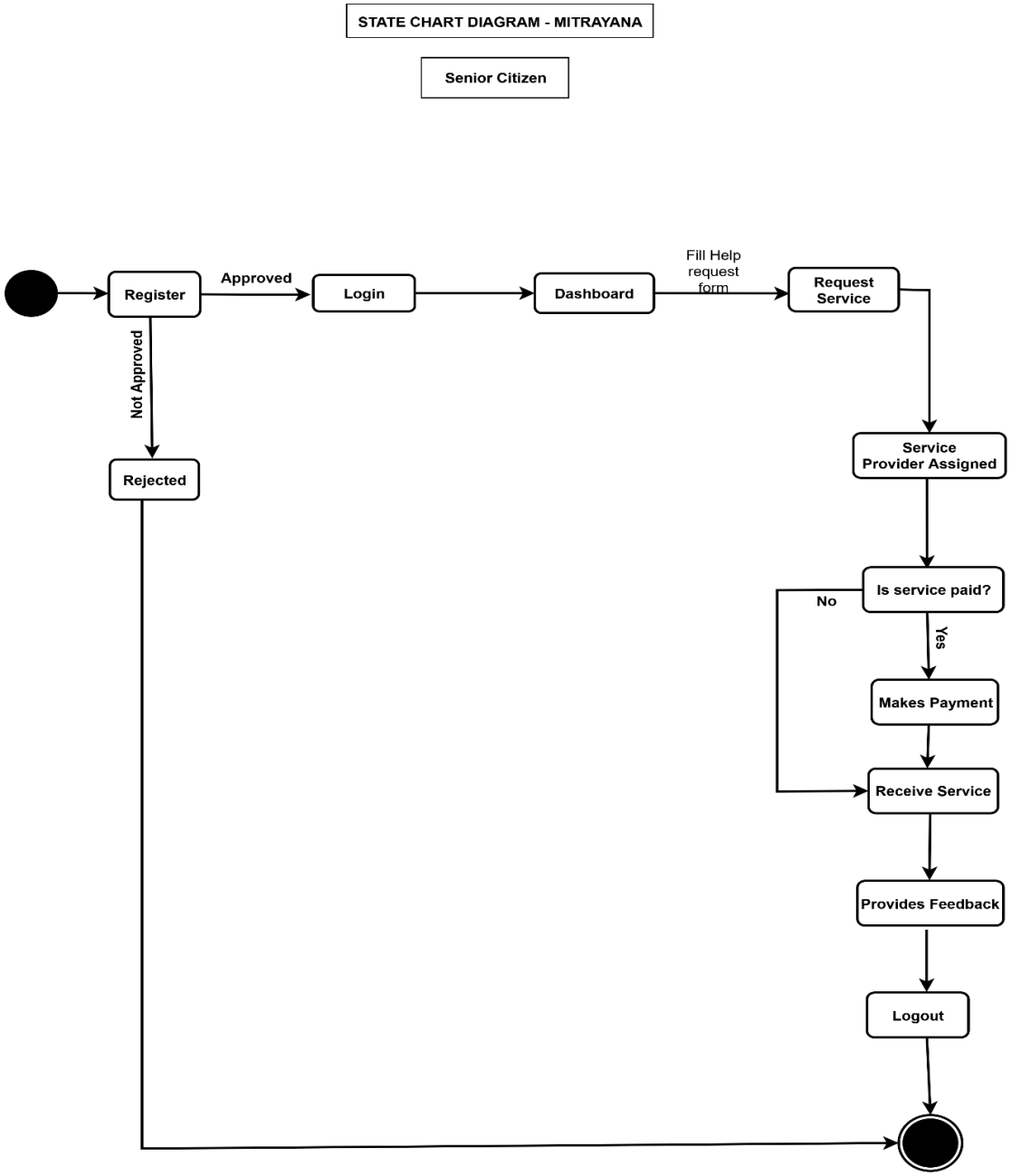
8.6.1 Admin State Chart Diagram



8.6.2 Service Provider State Chart Diagram



8.6.3 Senior Citizen State Chart Diagram



9.Data Dictionary

1. Admin:

Table Name: admin
Description: Stores admin login and control details.
Primary Key: admin_id
Foreign Key: --

Id	Field	Data Type	Constraints	Description
1.	admin_id	Int (10)	Primary key, Auto_Increment	Stores Unique ID for admin
2.	name	Char (50)	NOT NULL	Full name of admin
3.	email	Varchar (100)	Unique, NOT NULL	Stores email for admin login
4.	password	Varchar (255)	NOT NULL	Stores encypted password for admin login.
5.	contact_no	BIGINT (10)	Unique, NOT NULL	Stores contact number of admin.
6.	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Stores account creation timestamp

Example:

admin_id	name	email	password	contact_no	created_at
1	Adya Sharma	adya157@gmail.com	********	9123456789	2025-01-05 10:15:00

2. Service Provider:

Table Name: service_provider
Description: Stores details of registered volunteers/service providers.
Primary Key: provider_id
Foreign Key: --

Id	Field	Data Type	Constraints	Description
1.	provider_id	Int (10)	Primary key, Auto_Increment	Stores Unique ID for service providers.
2.	full_name	Char (50)	NOT NULL	Full name of service providers.
3.	email	Varchar (100)	NOT NULL	Stores email for provider's login
4.	password	Varchar (255)	NOT NULL	Stores encrypted password for provider's login.
5.	contact_no	BIGINT(10)	Unique, NOT NULL	Stores contact number of service providers.
6.	dob	Date	NOT NULL	Stores volunteer's date of birth.
7.	gender	Enum('male','female', 'other')	NOT NULL	Stores gender of providers.
8.	address	Text	NOT NULL	Stores address of provider's for matching it with senior citizen's request.
9.	city	Varchar (50)	NOT NULL	Stores city of service providers.
10	pincode	Int(6)	NOT NULL	Stores pin code for better accessibility.
11	availability	ENUM('available', 'busy','inactive')	DEFAULT 'available'	Shows current availability details.
12	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Stores account creation timestamp

Example:

full_name	email	password	contact_no	dob	gender	address	city	pincode	avail.	created_at
Rajesh Kumar	rajesh@gmail.com	********	9988776655	12-04-1995	male	Abrama road	Surat	394101	Available	01-02-2025 08:30
Sneha Iyer	Sneha23@gmail.com	********	9758368253	13-05-1992	female	Satellite	Ahmedabad	382001	Available	01-03-2025 00:00
Krishna Thaku	Krish@gmail.com	********	7872783652	19-04-1987	male	Residency road	Vadodara	390001	Busy	01-04-2025 00:00
Sheetal Shah	sheet351@gmail.com	********	9908738641	23-06-1999	female	Golf street	Vapi	396185	Inactive	01-05-2025 00:00
Rohini Patel	rohini21@gmail.com	********	9680796784	16-09-1990	female	University area	Ahmedabad	382001	Busy	01-06-2025 00:00

3. Senior Citizen:

Table Name: senior
Description: Stores details of elderly users.
Primary Key: senior_id
Foreign Key: --

Id	Field	Data Type	Constraints	Description
1.	senior_id	Int (10)	Primary key, Auto_Increment	Stores Unique ID for elderly users.
2.	full_name	Char (50)	NOT NULL	Full name of senior citizens.
3.	email	Varchar (100)	NOT NULL	Stores email for senior's login.
4.	password	Varchar (255)	NOT NULL	Stores encrypted password for senior's login.
5.	contact_no	BIGINT (10)	Unique, NOT NULL	Stores contact number of seniors.
6.	dob	Date	NOT NULL	Stores senior citizen's date of birth.
7.	gender	Enum('male','female', 'other')	NOT NULL	Stores gender of elderly.
8.	address	Text	NOT NULL	Stores address of senior's for matching it with provider's request.
9.	city	Varchar (50)	NOT NULL	Stores city of elderly users.
10	pincode	Int(6)	NOT NULL	Stores pin code for better accessibility.
11	blood_group	ENUM('A+','A-','B+','B-','AB+','AB-','O+','O-')	NULL	Stores blood group of the seniors.
12	medical_conditions	Text	Null	Stores data of medical conditions of the senior citizen(if any).
13	allergies	Text	Null	Stores data of allergic conditions of the senior citizen(if any).
14	emergency_contact	BIGINT(10)	Unique, NOT NULL	Stores emergency contact.
15	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Stores account creation timestampss

Example:

senior_id	full_name	email	password	contact_no	dob	gender	address	city	pincode	blood_group	medical_conditions	allergies	Emergency_contac	created_at
1	Sita Devi	sita10@gmail.com	********	9876543210	10-03-1950	Female	Satellite	Ahmedabad	382001	B+	Hypertension, Arthritis	Penicillin allergy	887654391C	01-02-2025 08:30
2	Mohan Patel	mohan11@gmail.com	********	9774533221	29-04-1951	Male	Abrama	Surat	382001	B+	Diabetes Type 2	None	8977542821	02-02-2025 09:15
3	Geeta Modi	geata90@gmail.com	********	8876543212	07-07-1952	Female	Vastral	Ahmedabad	382001	O+	Osteoporosis	Sulfa drugs	987555293C	01-02-2025 08:30
4	Arun Kumar	Kumar23@gmail.com	********	9776543203	10-02-1953	Male	Ranip	Ahmedabad	382001	O-	Asthma	Peanut allergy,	8574500040	02-02-2025 09:15
5	Sarita vaghela	sarita21@gmail.com	********	9874343214	20-12-1954	Female	Residency road	Vadodara	382001	AB+	Heart Disease	None	817751325C	03-02-2025 10:10
6	Rupesh Shah	rupesh10@gmail.com	********	9584522325	17-03-1955	Male	Golf Street	Vapi	382001	A+	Heart Disease	Lactose	981744396C	04-02-2025 10:10

4. Service:

Table Name: service
Description: Stores details of services offered by providers
Primary Key: service_id
Foreign Key: provider_id →service_provider(provider_id)

Id	Field	Data Type	Constraints	Description
1.	service_id	Int (10)	Primary key, Auto_Increment	Stores Unique service ID.
2.	provider_id	Int(10)	Foreign Key	Links service to providers.
3.	category	Varchar (100)	NOT NULL	Stores category of services provided.
4.	description	TEXT	NOT NULL	Stores details of the services offered.
5.	charge	DECIMAL(10,2)	Not null,CHECK(charge>=0)	Stores cost of the paid services.
6.	created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	The date on which service was listed

Example:

service_id	provider_id	category	description	charge	created_at
1	1	Household support	Help with grocery runs	150.01	03-02-2025 10:00
2	2	Health Assistance	Help with doctor visits	150.01	02-03-2025 10:20
3	3	Household support	Help with medicine pickup	100.01	02-04-2025 10:30
4	4	Wellness	Park visits	0	03-05-2025 17:25
5	5	Wellness	Play indoor games like ludo	0	01-06-2025 11:39

5. Help Request:

Table Name: help_request
Description: Stores requests raised by senior citizens.
Primary Key: request_id
Foreign Key: senior_id → senior(senior_id) provider_id → service_provider(provider_id)

Id	Field	Data Type	Constraints	Description
1.	request_id	Int (10)	Primary key, Auto_Increment	Stores Unique ID for requests made by senior citizens.
2.	senior_id	INT(10)	FOREIGN KEY	Links senior id with help request.
3.	provider_id	INT (10)	FK, NULL	Stores ID of assigned provider.
4.	request_type	ENUM ('grocery', 'medicine', 'companionship', ’other’)	NOT NULL	Stores type of request made by senior.
5.	description	TEXT	NOT NULL	Stores details of request made.
6.	service_type	ENUM('free', 'paid')	DEFAULT 'free'	Whether request is free or paid.
7.	request_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Stores information of the date and time at which the request was made.
8.	pincode	INT(6)	NOT NULL	Location from where the request was made.
9.	status	ENUM('pending', 'assigned', 'completed', 'cancelled')	DEFAULT 'pending'	Shows current status of help request made.

Example:

request_id	senior_id	provider_id	request_type	description	service_type	request_date	pincode
1	**1**	**1**	Household support	Need weekly groceries	paid	04-02-2025 09:45	382001
2	**2**	**2**	Wellness	Indoor games invite	free	05-02-2025 09:45	382007
3	**3**	**3**	Household support	Need weekly groceries	paid	06-02-2025 09:45	382001
4	**4**	**4**	Household support	Need help with laptop	paid	07-02-2025 09:45	382002
5	**5**	**5**	Wellness support	Visiting nearby park	free	08-02-2025 09:45	382001

6. Feedback:

Table Name: feedback
Description: Stores feedback exchanged between seniors and service providers.
Primary Key: feedback _id
Foreign Key: i. senior_id → senior(senior_id) ii. provider_id → service_provider(provider_id)

Id	Field	Data Type	Constraints	Description
1.	feedback_id	Int (10)	Primary key, Auto_Increment	Stores Unique feedback ID.
2.	senior_id	Int (10)	FK, Null	Links with senior id.
3.	provider_id	Int (10)	FK, Null	Links with provider id.
4.	feedback_by	ENUM('senior','provider')	NOT NULL	Mentions who gave the feedback.
5.	comments	TEXT	NULL	Stores comments given by user.
6.	rating	INT(1)	CHECK(rating BETWEEN 1 AND 5)	Rating value (1–5 stars)
7.	feedback_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time of the feedback submitted

Example:

feedback_id	senior_id	provider_id	feedback_by	comments	rating	feedback_date
1	1	1	senior	Very helpful with groceries	5	05-02-2025 12:00
2	2	2	senior	Very helpful with groceries	4	06-02-2025 13:20
3	3	3	senior	On-time delivery of medicines	3	07-02-2025 11:45
4	4	4	provider	Good experience with Mitrayana	5	08-02-2025 12:00
5	5	5	provider	Good experience with Mitrayana	5	09-02-2025 10:08

7. Payment:

Table Name: payment
Description: Stores details of cash payments handled via admin.
Primary Key: payment _id
Foreign Key: i. request_id → help_request(request_id) ii. senior_id → senior(senior_id) iii. provider_id → service_provider(provider_id)

Id	Field	Data Type	Constraints	Description
1.	payment_id	Int (10)	Primary key, Auto_Increment	Stores Unique payment ID.
2.	request_id	Int (10)	FK	Links withhelp request.
3.	provider_id	Int (10)	FK	Links with provider id.
4.	senior_id	Int (10)	FK	Mentions the senior who gave payment.
5.	amount	DECIMAL(10,2)	NOT NULL, CHECK(amount >= 0)	Stores payment amount.
6.	method	ENUM('cod')	NOT NULL	Payment made in cash.
7.	status	ENUM('pending', 'received', 'verified')	DEFAULT 'pending'	Shows payment status.
8.	admin_verified	BOOLEAN	DEFAULT 0	1 = verified by admin
9.	paid_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time stamp when payment was recorded

Example:

payment_id	request_id	provider_id	senior_id	amount	method	status	admin_verified	paid_at
1	1	1	1	150.01	cod	verified	1	05-02-2025 12:00
2	2	2	2	200.01	cod	verified	1	05-03-2025 13:15
3	3	3	3	400.01	cod	received	1	05-04-2025 10:20
4	4	4	4	0	cod	received	1	06-04-2025 10:20
5	5	5	5	0	cod	received	1	05-06-2025 17:05